COM3307 · EMBEDDED SYSTEMS · 3 CREDITS · EXAM: 3 HOURS

Embedded Systems – Exam Notes

Made from the 2025 and 2026 past papers, because the lecture notes were not in your folder. Every section ends with the past-paper questions it answers. English is written the way you can write it in the exam; the සිංහල boxes explain the idea simply.

---

0. How this paper works
1. What is an embedded system?
2. Applications, microcontroller vs microprocessor
3. Requirements and real-time systems
4. Hardware components
5. Communication: UART, SPI, I2C
6. Polling, interrupts and ISRs
7. Embedded C: functions, ports, LED + button
8. Hardware timers and a 1-second delay
9. AVR assembly and status flags
10. Microcontroller architecture, fetch–decode–execute
11. Design scenario questions (Q2 every year)
12. Past paper map

---

0. How this paper works

- **Five questions, 3 hours.** Questions 1 and 2 are **compulsory**. Answer **any two** of Questions 3–5. Each question is about 25 marks.
- **Q1** is always definitions and comparisons: embedded system definition and characteristics, applications, microcontroller vs microprocessor, functional vs non-functional requirements.
- **Q2** is always a design scenario: smart waste bin (2025), smart street lighting (2026). You list sensors, controller and actuators, write the logic, and discuss challenges or improvements.
- **Q3–Q5** rotate: components and communication, interrupts and embedded C, timers and real-time (2025); AVR assembly, ports, fetch–decode–execute and architecture (2026).

> **Plan:** Master sections 1–4 and 11 first (they cover Q1 and Q2 = half the paper). Then make two of these strong: **AVR assembly** (section 9), **timers + interrupts** (sections 6 and 8), **architecture** (section 10).

> **සිංහලෙන්:** ප්‍රශ්න 5යි. 1 සහ 2 අනිවාර්යයි. 3, 4, 5 න් ඕනෑම දෙකක් කරන්න. Q1 හැමදාම definitions සහ comparisons. Q2 හැමදාම "design එකක් කරන්න" ප්‍රශ්නයක් (smart bin, street light වගේ). ඉතින් මුලින්ම section 1–4 සහ 11 හොඳට ඉගෙන ගන්න – ඒකෙන් paper එකෙන් බාගයක් cover වෙනවා.

1. What is an embedded system?

Definition. An embedded system is a combination of computer hardware and software, designed to perform a **specific, dedicated function** within a larger mechanical or electrical system. It usually runs a fixed program (firmware) and interacts with the physical world through sensors and actuators.

Examples: washing machine controller, microwave oven, digital watch, car airbag controller, pacemaker, traffic light controller, smart meter.

Key characteristics (how it differs from a general-purpose computer)

1. **Dedicated function.** It does one job (or a small set of jobs) for its whole life. A PC runs any program the user installs.
2. **Real-time operation.** It must react within a fixed time limit (an airbag must fire within milliseconds).
3. **Resource-constrained.** Small memory (KB, not GB), slow CPU (MHz), limited power and low cost.
4. **Reactive and connected to the physical world.** It reads sensors and drives actuators continuously.
5. **High reliability and stability.** It must run for months or years without restarting, often with no human watching.
6. **Low power consumption.** Many run on batteries, so they sleep whenever possible.
7. **Fixed firmware.** The program is stored in ROM/Flash; the user normally cannot change it.
8. **Small size, little or no user interface** (a few buttons, LEDs or a small display).

In the exam "explain three characteristics" = write three of these, each with one sentence of explanation and one example.

Feature	Embedded system	General-purpose computer
Purpose	One dedicated task	Many tasks, any software
Hardware	Microcontroller, small memory	Powerful processor, GBs of RAM
Software	Fixed firmware, maybe an RTOS	Full OS (Windows, Linux), user installs apps
Timing	Real-time deadlines	Best effort
Power and cost	Very low	High
User interface	Minimal or none	Keyboard, mouse, large screen
Example	Washing machine controller	Laptop

සිංහලෙන්: Embedded system එකක් කියන්නේ **එක නිශ්චිත වැඩක්** කරන්න හදපු පොඩි computer එකක් – ඒක ලොකු machine එකක් ඇතුළේ "embed" වෙලා (හිරවෙලා) තියෙන්නේ. උදා: washing machine එකේ program එක control කරන chip එක.

Laptop එකකට ඕනෑම software එකක් දාන්න පුළුවන්. ඒත් washing machine controller එක කරන්නේ රෙදි හෝදන එක විතරයි. ඒකට memory ටිකයි, speed අඩුයි, current අඩුවෙන් ගන්නවා, ලාභයි, දවස් ගණන් restart නොකර වැඩ කරනවා, වෙලාවට response කරන්න ඕනේ (real-time).

Past papers ▸ Answers: 2025 Q1(a) 10 marks · 2026 Q1(a) 6 marks · 2025 ICT Q1(a)

---

2. Applications, microcontroller vs microprocessor

Real-world applications and their impact

Area	Example embedded systems	Impact
Healthcare	Pacemaker, insulin pump, patient monitor, digital thermometer	Continuous monitoring, saves lives, home care
Automotive	Engine control unit (ECU), ABS brakes, airbag controller, reversing sensor	Safety, fuel efficiency, lower emissions
Home and consumer	Washing machine, microwave, smart TV, smart thermostat	Convenience, energy saving
Industry	PLCs, robot arms, conveyor control	Higher productivity, precision, safer workplaces
Agriculture	Smart irrigation with soil-moisture sensors	Saves water and labour, better yield
Communication	Routers, mobile phones, smart meters	Connectivity, remote monitoring

"Describe three applications and their impact" = pick three rows; for each write what the system does + one impact sentence.

Microcontroller vs microprocessor

Microcontroller (MCU): a single chip that contains a CPU **plus** RAM, ROM/Flash, I/O ports, timers, ADC and serial interfaces. It is a complete small computer for a dedicated task. Examples: ATmega328P (Arduino Uno), PIC16F877A, 8051, ESP32.

Microprocessor (MPU): a chip that contains **only the CPU**. RAM, storage and I/O must be added as separate chips. It is built for high processing power. Examples: Intel Core i5, AMD Ryzen, ARM Cortex-A in phones.

Point	Microcontroller	Microprocessor
On the chip	CPU + memory + I/O + timers	CPU only
External parts needed	Very few	RAM, storage, I/O chips
Cost, size, power	Low	High
Speed	MHz range	GHz range
Used for	Dedicated control tasks (embedded)	General computing (PCs, laptops)

සිංහලෙන්: Microcontroller එකේ CPU එකයි, memory එකයි, input/output pins ටිකයි, timers ටිකයි ඔක්කොම **එකම chip එකේ** තියෙනවා – ඒ නිසා ලාභයි, පොඩියි, current අඩුවෙන් යනවා (උදා: Arduino එකේ ATmega328P). Microprocessor එකේ තියෙන්නේ CPU එක විතරයි – RAM, hard disk වගේ දේවල් වෙනම සවි කරන්න ඕනේ, ඒත් ගොඩක් වේගවත් (උදා: laptop එකේ Intel Core i5).

Past papers ▸ Answers: 2025 Q1(b) 10 marks, Q1(c) 5 marks · 2025 ICT Q1(b)(c)

---

3. Requirements and real-time systems

Functional vs non-functional requirements

Functional requirements say **what** the system must do – its functions and behaviour.

Non-functional requirements say **how well** it must do it – quality limits such as response time, power, cost, size, reliability, accuracy and security.

Example: a smart home temperature monitor

- Functional: read the room temperature every 5 seconds; switch the fan on when the temperature is above 30 °C; show the temperature on an LCD; send an alert to the phone if it is above 35 °C.
- Non-functional: respond within 1 second; accuracy ±0.5 °C; run 24/7 without failure; battery lasts at least 6 months; cost below Rs. 5,000.

Hard vs soft real-time systems

A **real-time system** must produce the correct result **within a deadline**. A late answer is a wrong answer.

Hard real-time: missing a deadline causes system failure, damage or danger to life. Examples: airbag controller, ABS braking, pacemaker, aircraft flight control.

Soft real-time: missing a deadline only reduces quality; the system keeps working. Examples: video streaming (a frame is dropped), online games, music players, a smart TV menu.

Point	Hard real-time	Soft real-time
Deadline miss	Not allowed – failure	Allowed sometimes – lower quality
Design	Worst-case timing must be guaranteed	Average performance is enough
Example	Airbag must fire within about 30 ms	Video call can skip a frame

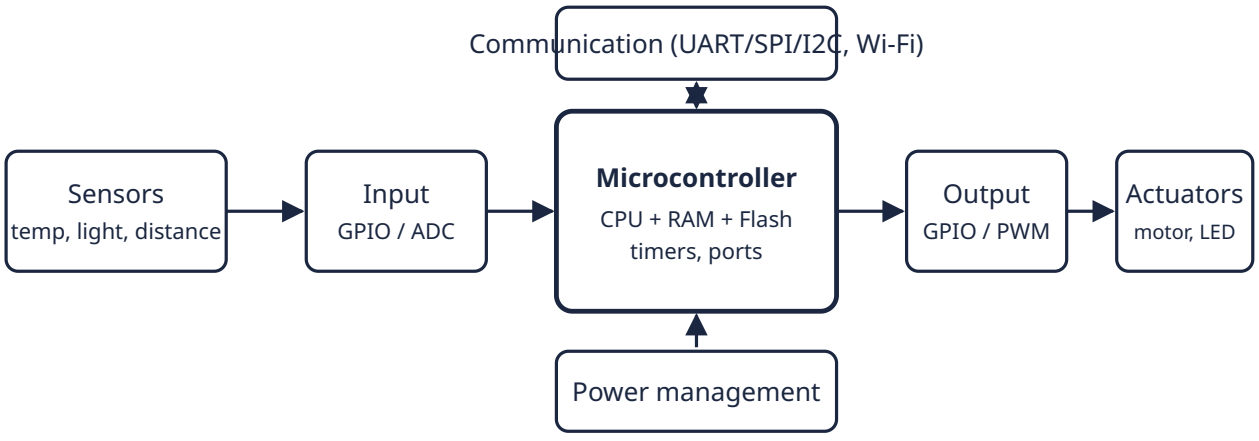
සිංහලෙන්: Functional = system එක **කරන්න ඕනේ වැඩ** (temperature කියවන්න, fan එක on කරන්න). Non-functional = ඒ වැඩේ **කොච්චර හොඳට** කරන්න ඕනෙද (තත්පර 1කින් response, battery මාස 6ක්, මිල අඩු).

Hard real-time එකේ deadline එක miss වුණොත් ලොකු අනතුරක් (airbag එක පරක්කු වුණොත් මිනිහා මැරෙන්න පුළුවන්). Soft real-time එකේ miss වුණොත් quality එක ටිකක් අඩු වෙනවා විතරයි (video එකේ frame එකක් පනිනවා).

Past papers ▸ Answers: 2026 Q1(b) 8 marks · 2025 Q5(c) 5 marks · 2025 ICT Q5(c)

---

4. Hardware components of an embedded system



Basic block diagram of an embedded system – draw this in any "components" or design answer.

Microcontroller

The brain. Runs the program, reads inputs, takes decisions and controls the outputs.

Memory (RAM & ROM)

Flash/ROM (program memory) stores the firmware permanently – it is non-volatile. **RAM/SRAM** (data memory) holds variables and the stack while running – it is volatile. **EEPROM** stores small settings that must survive power loss (e.g. calibration values).

I/O interfaces

Connect the MCU to the outside world: digital GPIO pins, **ADC** (converts an analog sensor voltage into a number), **PWM** (controls motor speed or LED brightness), communication ports.

Sensors

Convert a physical quantity into an electrical signal. Examples: LM35 / DHT11 temperature, LDR light, HC-SR04 ultrasonic distance, PIR motion, soil moisture, MQ-2 gas.

Actuators

Convert an electrical signal into physical action. Examples: DC/servo/stepper motors, relays, LEDs, buzzers, solenoid valves, displays.

Communication modules

Exchange data with other chips or systems: UART, SPI, I2C (wired, on the board); Wi-Fi, Bluetooth, GSM, LoRa, Zigbee (wireless).

Power management unit

Regulates the voltage, manages the battery or solar charging, switches the MCU into sleep modes and protects against low voltage (brown-out). Gives long battery life and stable operation.

> **සිංහලෙන්:** Sensor = ලෝකයේ දෙයක් (උෂ්ණත්වය, එළිය, දුර) **electrical signal** එකක් කරනවා. Microcontroller = ඒ signal එක කියවලා **තීරණය** ගන්නවා. Actuator = තීරණය අනුව **වැඩක් කරනවා** (motor එක කරකවනවා, LED එක පත්තු කරනවා). Flash එකේ program එක ස්ථිරව තියෙනවා; RAM එකේ run වෙන වෙලාවේ variables තියෙනවා (current ගියාම මැකෙනවා). Power unit එක battery එක ඉතුරු කරනවා.

Past papers ▸ Answers: 2025 Q3(a) 9 marks (explain any three) · used in every Q2 design answer

5. Communication: synchronous vs asynchronous, UART, SPI, I2C

Synchronous communication: the sender and receiver share a **common clock signal**. Each bit is read on a clock edge, so no start/stop bits are needed and it can be fast. Needs an extra clock wire. Examples: **SPI**, **I2C**.

Asynchronous communication: there is **no shared clock**. Both sides agree on a speed (baud rate, e.g. 9600 bps) and every byte is wrapped with a **start bit** and **stop bit(s)**, sometimes a parity bit. Example: **UART** – an Arduino talking to a PC, a GPS module or a Bluetooth HC-05 module.

UART frame (one byte): line idle = 1 → start bit 0 → 8 data bits (LSB first) → optional parity → stop bit 1. Wires: TX, RX, GND. Point-to-point, full duplex.

	UART	SPI	I2C
Type	Asynchronous	Synchronous	Synchronous
Wires	2 (TX, RX)	4 (SCLK, MOSI, MISO, SS)	2 (SDA, SCL)
Devices	2 (point-to-point)	1 master, many slaves (one SS line each)	Many masters and slaves, 7-bit addresses
Duplex	Full	Full	Half
Speed	Slow–medium (9600–115200 bps)	Fastest (MHz)	100 kHz / 400 kHz
Typical use	PC serial, GPS, Bluetooth, GSM	SD card, TFT display, flash memory	Sensors, RTC, small OLED displays

සිංහලෙන්: Synchronous වල දෙපැත්තටම පොදු **clock** wire එකක් තියෙනවා – clock එක "tick" වෙන හැම වෙලාවකම bit එකක් කියවනවා. Asynchronous වල clock එකක් නෑ – දෙන්නම එකම speed (baud rate) එකකට එකඟ වෙනවා; හැම byte එකටම මුලට start bit එකකුයි අගට stop bit එකකුයි දානවා, එතකොට byte එක පටන් ගත්තේ කොහෙන්ද කියලා receiver ට තේරෙනවා. UART = async. SPI, I2C = sync.

Past papers ▸ Answers: 2025 Q3(b) 8 marks · 2025 Q3(a) "communication modules"

6. Polling, interrupt-driven I/O and ISRs

Polling: the CPU repeatedly checks (in a loop) whether a device needs service, e.g. "is the button pressed?" again and again. Simple to write, but it wastes CPU time and power, and a very short event can be missed or handled late.

Interrupt-driven I/O: the device sends an **interrupt signal** to the CPU only when it needs attention. The CPU pauses the main program, runs a special function (the ISR), and then continues where it stopped. The CPU is free (or asleep) the rest of the time, and the response is fast.

A scenario where interrupts are preferred

An **emergency-stop button** on a factory machine: the motor must stop immediately, even if the main program is busy. With polling, the button might be checked too late. With an external interrupt (INT0), the CPU jumps to the stop routine within microseconds.

Other good scenarios: a battery-powered sensor node that sleeps until a PIR motion sensor wakes it; receiving serial data (UART RX interrupt) so no byte is lost; counting fast pulses from a wheel encoder; an airbag crash sensor.

Interrupt Service Routine (ISR)

An **ISR** is a special function that the processor runs **automatically** when a specific interrupt occurs. It handles the event and then returns control to the interrupted program.

Purpose: respond quickly to time-critical events; handle hardware events such as a timer overflow, a button press or received data; let the CPU do other work (or sleep) instead of waiting; save power.

What happens when an interrupt occurs:

1. The device sets its interrupt flag.
2. The CPU finishes the current instruction.
3. It saves the return address (program counter) on the stack and disables further interrupts.
4. It jumps to the ISR address found in the **interrupt vector table**.
5. The ISR runs.
6. The `RETI` instruction restores the program counter, re-enables interrupts and the main program continues.

Rules for a good ISR: keep it short; no delays or long loops inside; use `volatile` variables to share data with the main program; clear the flag if the hardware does not.

```
// ATmega328P: toggle an LED on PB0 when a button on INT0 (PD2) is pressed
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(INT0_vect) {            // runs automatically on the interrupt
    PORTB ^= (1 << PB0);    // toggle LED
}

int main(void) {
    DDRB  |= (1 << PB0);    // LED pin = output
    PORTD |= (1 << PD2);    // pull-up on the button pin
    EICRA |= (1 << ISC01);  // interrupt on falling edge (button press)
    EIMSK |= (1 << INT0);   // enable INT0
    sei();                  // enable global interrupts
    while (1) { }           // main program is free to do other work
}
```

> **සිංහලෙන්:** Polling කියන්නේ CPU එක නැවත නැවත "button එක press කළාද? press කළාද?" කියලා අහන එක – මහන්සියි, current නාස්තියි. Interrupt කියන්නේ button එකම "මට දැන් උදව් ඕනේ" කියලා CPU එකට කියන එක. එතකොට CPU එක කරමින් හිටපු වැඩේ නවත්තලා **ISR** එක run කරනවා, ඊට පස්සේ ආයෙත් කලින් වැඩේට යනවා. ISR එක කෙටි වෙන්න ඕනේ.

Past papers ▸ Answers: 2025 Q3(c) 4 marks, Q3(d) 4 marks, Q4(c) 5 marks · 2025 ICT same

---

7. Embedded C: functions, port registers, LED + button

Why functions are useful in embedded C

1. **Modularity:** a big program is split into small tasks (`readSensor()` , `updateDisplay()`).
2. **Reusability:** write once, call many times (`delay_ms(500)` used everywhere).
3. **Smaller code size:** no repeated code, so less Flash memory is used – important on small MCUs.
4. **Readability and maintenance:** `led_on()` is easier to understand than a bit operation.
5. **Easier testing and debugging:** each function can be tested alone.
6. **Hides hardware details:** if the LED moves to another pin, only one function changes.

```
void led_on(void)  { PORTB |=  (1 << PB0); }   // set bit = LED ON
void led_off(void) { PORTB &= ~(1 << PB0); }   // clear bit = LED OFF
uint8_t button_pressed(void) {                  // button to GND, pull-up on
    return !(PIND & (1 << PD2));                // pressed reads 0
}
```

AVR port registers

Register	Meaning	Example
`DDRx`	Data Direction Register: 1 = output, 0 = input	`DDRB = 0xFF;` all PORTB pins output
`PORTx`	Output value (1 = high, 0 = low). On an input pin, 1 turns on the internal pull-up	`PORTB = 0xAA;`
`PINx`	Reads the actual logic level on the pins	`if (PIND & (1 << PD2))`

Turn ON an LED when a button is pressed

Assumptions (always write them): LED on PB0 with a resistor (active high). Push button between PD2 and GND, internal pull-up enabled, so pressed = 0.

```
#include <avr/io.h>

int main(void) {
    DDRB  ⊨  (1 << PB0);      // PB0 output (LED)
    DDRD  &= ~(1 << PD2);     // PD2 input (button)
    PORTD ⊨  (1 << PD2);      // enable pull-up

    while (1) {
        if (!(PIND & (1 << PD2)))     // button pressed (reads 0)
            PORTB ⊨  (1 << PB0);     // LED ON
        else
            PORTB &= ~(1 << PB0);     // LED OFF
    }
}
```

Pseudocode

```
START
  set LED pin as OUTPUT
  set button pin as INPUT with pull-up
  LOOP forever
     read button pin
     IF button is pressed THEN LED = ON
     ELSE LED = OFF
  END LOOP
```

Flow diagram (draw as boxes): Start → Configure pins → Read button → ◇ Pressed? → Yes: LED ON / No: LED OFF → back to Read button.

සිංහලෙන්: `DDRx` = pin එක input ද output ද කියලා කියන register එක (1 = output). `PORTx` = output pin එකට 1 ද 0 ද යවන්නේ කියන එක. `PINx` = input pin එකේ දැන් තියෙන අගය කියවන එක. Button එක GND එකට සම්බන්ධ කරලා pull-up එක on කළාම, press කළාම 0 කියවෙනවා – ඒකයි `!` (NOT) එක දාලා තියෙන්නේ. Flow chart හෝ pseudocode ලිව්වත් marks ලැබෙනවා – ප්‍රශ්නයේම එහෙම කියනවා.

Past papers ▸ Answers: 2025 Q4(a) 10 marks, Q4(b) 10 marks · 2026 Q4(b) 15 marks (with PORTD → PORTB)

---

8. Hardware timers and a 1-second delay

A **hardware timer** is a counter register inside the microcontroller that counts clock pulses **independently of the CPU**. The clock first passes through a **prescaler** (÷1, ÷8, ÷64, ÷256, ÷1024), then the counter register (`TCNT0`) increases by one on every tick.

An **8-bit timer** counts 0 → 255. On the next tick it **overflows** back to 0 and sets the overflow flag, which can trigger an interrupt. In **CTC mode** (Clear Timer on Compare) the counter is compared with `OCR0A` ; when they are equal the counter resets to 0 and a compare-match interrupt can run.

Delay generation: because the tick time is fixed, counting a known number of ticks gives an exact time. The CPU can either wait for the flag (polling) or keep working and let the timer interrupt count the time.

```
Tick time = prescaler ÷ clock frequency
Overflow time (8-bit) = 256 × tick time
```

Worked example: 1-second delay, 8-bit timer, 1 MHz clock (2025 Q5(b))

Prescaler	Tick time	Overflow time (256 ticks)	Overflows in 1 s
1	1 µs	0.256 ms	3906.25
8	8 µs	2.048 ms	488.28
64	64 µs	16.384 ms	61.04
1024	1.024 ms	262.144 ms	3.81

Exact method (CTC): prescaler 8 → tick = 8 ÷ 1 MHz = 8 µs. For 1 ms we need 1 ms ÷ 8 µs = **125 ticks**, so set `OCR0A = 124` (counts 0…124). The compare interrupt now runs every 1 ms. Count **1000** interrupts = **exactly 1.000 s**.

Simple overflow method: prescaler 64 → overflow every 16.384 ms. 1 s ÷ 16.384 ms = 61.04, so count **61 overflows** ≈ 0.9994 s (0.06 % error).

```
// ATmega328P, 1 MHz clock: toggle an LED on PB5 every 1 second
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint16_t ms = 0;

ISR(TIMER0_COMPA_vect) {          // runs every 1 ms
    ms++;
    if (ms ⩾ 1000) {              // 1000 ms = 1 s
        ms = 0;
        PORTB ^= (1 << PB5);      // 1-second event
    }
}

int main(void) {
    DDRB  ⊨ (1 << PB5);           // LED output
    TCCR0A = (1 << WGM01);        // CTC mode
    OCR0A  = 124;                 // 125 ticks x 8 us = 1 ms
    TIMSK0 = (1 << OCIE0A);       // enable compare-match interrupt
    TCCR0B = (1 << CS01);         // prescaler 8 → timer starts
    sei();                        // global interrupts on
    while (1) { }                 // CPU free for other work
}
```

Pseudocode / flow (enough for full marks if you explain the numbers):

```
MAIN: set LED pin output
      set timer: prescaler 8, CTC mode, compare value 124
      enable compare interrupt, enable global interrupts
      LOOP forever (do other work)

ISR (every 1 ms): count = count + 1
      IF count = 1000 THEN  toggle LED (1 second passed), count = 0
      RETURN
```

Register names are for the ATmega328P (Arduino Uno). Other AVR chips use slightly different names (e.g. ATmega32 uses `TCCR0`, `TIMSK`); the idea is the same.

> **සිංහලෙන්:** Timer එක කියන්නේ CPU එකෙන් ස්වාධීනව clock pulses ගණන් කරන counter එකක්. Prescaler එකෙන් clock එක බෙදනවා (÷8 නම් 1 MHz → tick එකක් 8 µs). 8-bit timer එක 255 දක්වා ගණන් කරලා ආයෙත් 0 ට යනවා (overflow). අපිට ඕනේ තත්පර 1ක් නම්: 8 µs × 125 = 1 ms; ඒ 1 ms interrupt එක 1000 පාරක් ගණන් කළාම හරියටම 1 s. Exam එකේ ගණනය (tick time, ticks ගණන, interrupts ගණන) පැහැදිලිව පෙන්නන්න – marks වැඩිපුරම එතන.

Past papers ▸ Answers: 2025 Q5(a) 10 marks, Q5(b) 10 marks · 2025 ICT same

9. AVR assembly and status flags

- AVR is an **8-bit RISC** microcontroller with **32 general-purpose registers**, R0–R31. `LDI` works only with **R16–R31**.
- Most instructions take 1 clock cycle. It uses a **Harvard architecture** (separate program and data memory).

Instruction	Meaning
`LDI Rd, K`	Load an immediate number K into Rd (R16–R31)
`MOV Rd, Rr`	Copy Rr into Rd
`ADD / SUB Rd, Rr`	Rd = Rd + Rr / Rd = Rd - Rr
`INC / DEC Rd`	Add 1 / subtract 1
`OUT A, Rr`	Copy a register to an I/O register (e.g. PORTB)
`IN Rd, A`	Copy an I/O register (e.g. PIND) into a register
`SBI / CBI A, b`	Set / clear one bit of an I/O register
`SBIS / SBIC A, b`	Skip the next instruction if the bit is set / cleared
`RJMP k`	Jump to label k
`BRNE / BREQ k`	Branch if Z = 0 (not equal) / if Z = 1 (equal)

Status register (SREG) flags

Flag	Set (=1) when…
C Carry	an addition carries out of bit 7, or a subtraction needs a borrow
Z Zero	the result is 0
N Negative	bit 7 of the result is 1
V Overflow	a signed (two's complement) overflow happened
S Sign	S = N XOR V
H Half carry	a carry/borrow between bit 3 and bit 4
I Interrupt	global interrupts are enabled (`SEI`)

2026 Q3(a): final value of r16

```
ldi r16, 0x05    ; r16 = 5
ldi r17, 0x03    ; r17 = 3
sub r16, r17     ; r16 = 5 - 3
```

r16 = 0x02 (binary 0000 0010).

SUB can change the flags **H, S, V, N, Z, C**. Here: **Z = 0** (result is not zero), **C = 0** (5 ≥ 3, no borrow), **N = 0** (bit 7 is 0), **V = 0** (no signed overflow), **S = N ⊕ V = 0**, **H = 0** (low nibble 5 ≥ 3, no borrow from bit 4). LDI does not change any flag.

2026 Q3(b): what does this loop do?

```
      ldi  r18, 0x03   ; counter = 3
Loop: dec  r18         ; counter = counter - 1
      brne Loop        ; repeat while counter ≠ 0 (Z = 0)
```

It is a **count-down loop** that runs **3 times** and is used as a short **software delay**.

- Pass 1: r18 = 3 → 2, Z = 0 → branch back.
- Pass 2: r18 = 2 → 1, Z = 0 → branch back.
- Pass 3: r18 = 1 → 0, **Z = 1** → BRNE does not branch; the program continues with the next instruction.

DEC changes S, V, N, Z (not C). Time: LDI 1 cycle + 3 × DEC (1) + 2 taken BRNE (2 each) + 1 not-taken BRNE (1) = **9 cycles** = 9 µs at 1 MHz. A bigger starting value gives a longer delay.

2026 Q4(a): PORTB as output, send 0xAA

```
ldi r16, 0xFF     ; 1111 1111
out DDRB, r16     ; all PORTB pins = output
ldi r16, 0xAA     ; 1010 1010
out PORTB, r16    ; PB7, PB5, PB3, PB1 = HIGH; PB6, PB4, PB2, PB0 = LOW
```

2026 Q4(b): switch on PORTD turns ON an LED on PORTB

Assumptions: switch on PD0 to GND with the pull-up on (pressed = 0); LED on PB0 (active high).

```
        ldi  r16, 0x01
        out  DDRB, r16     ; PB0 output (LED)
        ldi  r16, 0x00
        out  DDRD, r16     ; PORTD input (switch)
        ldi  r16, 0x01
        out  PORTD, r16    ; pull-up on PD0
MAIN:   sbis PIND, 0       ; if PD0 = 1 (not pressed) skip next line
        rjmp LED_ON        ; PD0 = 0 → pressed
        cbi  PORTB, 0      ; LED OFF
        rjmp MAIN
LED_ON: sbi  PORTB, 0      ; LED ON
        rjmp MAIN
```

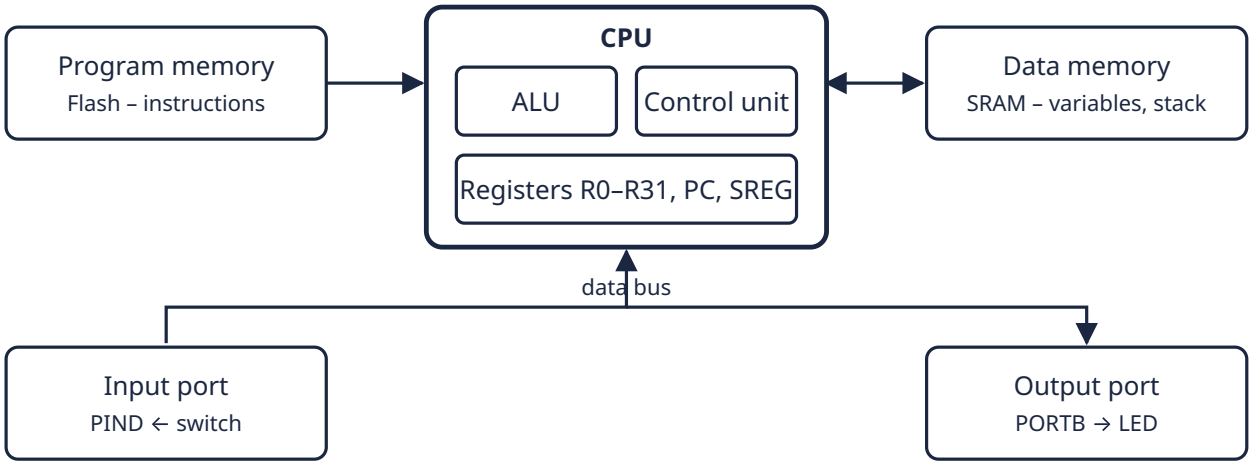
Pseudocode version: set PB0 as output, PORTD as input → loop: read PD0 → if pressed, PB0 = 1 else PB0 = 0 → repeat.

> **සිංහලෙන්:** AVR වල registers 32ක් (R0–R31). `LDI` කරන්න පුළුවන් R16–R31 වලට විතරයි. `SUB` එකෙන් පස්සේ flags බලන්න: result එක 0 ද? (Z), borrow එකක් ආවද? (C), bit 7 = 1 ද? (N). 5 - 3 = 2 නිසා ඔක්කොම flags 0යි. `DEC` + `BRNE` loop එකේ r18, 3 → 2 → 1 → 0 වෙනවා; 0 වුණාම Z = 1 වෙලා loop එකෙන් එළියට යනවා – ඒ කියන්නේ loop එක 3 පාරක් යනවා, පොඩි delay එකක්.

Past papers ▸ Answers: 2026 Q3(a) 12 marks, Q3(b) 13 marks, Q4(a) 10 marks, Q4(b) 15 marks

---

10. Microcontroller architecture and the fetch–decode–execute cycle



Simple microcontroller model (Harvard: program and data memory on separate buses).

CPU

ALU does arithmetic and logic; the **control unit** decodes instructions and sends control signals; **registers** hold the data being worked on. The **program counter (PC)** holds the address of the next instruction; **SREG** holds the flags.

Program memory (Flash)

Stores the program instructions; non-volatile.

Data memory (SRAM)

Stores variables and the stack while the program runs; volatile.

I/O ports

Connect pins to the outside: inputs (switches, sensors) and outputs (LEDs, motors).

Fetch–decode–execute

1. **Fetch:** the CPU reads the instruction at the address in the PC from program memory into the instruction register; the PC increases to point to the next instruction.

2. **Decode:** the control unit works out the operation (opcode) and the operands (which registers or constant).
3. **Execute:** the ALU performs the operation, the result is written back to a register or memory, and the flags in SREG are updated.

Example: `ADD R16, R17` with R16 = 5 and R17 = 3. Fetch: the instruction is read from Flash at address PC; PC = PC + 1. Decode: "ADD, destination R16, source R17". Execute: the ALU adds 5 + 3 = 8, writes 8 into R16, sets Z = 0, C = 0, N = 0.

AVR overlaps these steps (a 2-stage pipeline): while one instruction executes, the next is being fetched, so most instructions finish in one clock cycle.

Data flow: input port → CPU → output port (2026 Q5(b))

1. **Read from an input port:** `IN R16, PIND` – the switch level on the pin is copied over the data bus into register R16.
2. **Processed by the CPU:** e.g. `ANDI R16, 0x01` or a compare – the ALU tests the bit; the control unit decides what to do next (branch).
3. **Sent to an output port:** `OUT PORTB, R16` – the register value travels over the data bus to the PORTB latch and appears on the pins, switching the LED on.

> **සිංහලෙන්:** CPU එක instruction එකක් run කරන්නේ පියවර 3කින්: (1) **Fetch** – PC එකේ address එකෙන් instruction එක Flash එකෙන් ගන්නවා, PC එක එකකින් වැඩි කරනවා; (2) **Decode** – මේ instruction එක මොකක්ද (ADD ද SUB ද), කොයි registers ද කියලා තේරුම් ගන්නවා; (3) **Execute** – ALU එකෙන් ගණනය කරලා result එක register එකට දානවා, flags update කරනවා. Diagram එකේ boxes 5ම (CPU, program memory, data memory, registers, I/O ports) අනිවාර්යයෙන් ඇඳලා, ඊතලවලින් data යන පාර පෙන්නන්න.

Past papers ▸ Answers: 2026 Q5(a) 10 marks, Q5(b) 15 marks

---

11. Design scenario questions (Q2 every year)

> **Answer template – use these headings:** 1) Assumptions · 2) Inputs / sensors (and why) · 3) Controller · 4) Outputs / actuators · 5) Communication · 6) Power · 7) Logic (pseudocode or flowchart) · 8) Challenges with solutions / improvements.

A. Smart waste bin that sends an alert when full (2025 Q2)

Assumptions: street bin 100 cm deep; "full" = 80 % or more; the city office receives alerts through GSM SMS or Wi-Fi.

- **Sensors:** HC-SR04 ultrasonic sensor under the lid measures the distance to the garbage (fill level). Optional load cell (weight) and gas sensor (smell).

- **Controller:** ESP32 (built-in Wi-Fi, low-power sleep) or ATmega328P + SIM800L GSM module.
- **Actuators / outputs:** LED indicator (green/red), buzzer, alert message to the collection team; optional servo motor to lock the lid when full.
- **Power:** battery + small solar panel; MCU sleeps between readings.

```
LOOP every 10 minutes
   d = average of 5 ultrasonic readings          (removes noise)
   fill% = (binDepth - d) / binDepth x 100
   IF fill% ⩾ 80 AND alertSent = false THEN
        send "Bin #12 full" message; LED = RED; alertSent = true
   ELSE IF fill% < 30 THEN                        (bin was emptied)
        alertSent = false; LED = GREEN
   go to deep sleep until next reading
END LOOP
```

Challenge	Solution
Wrong readings from uneven garbage or dirt on the sensor	Average/median of several readings; mount the sensor at the top centre; add a weight sensor as a second check
No mains power on the street	Solar panel + battery, deep-sleep mode
Weak network coverage	GSM or LoRa; store the alert and retry until sent
Rain, heat, vandalism	Waterproof (IP65) sealed enclosure, tamper-proof mounting

B. Smart street lighting (2026 Q2)

- **Sensors:** LDR (ambient light level, read through the ADC); PIR motion sensor (people or vehicles nearby). Optional real-time clock (RTC).
- **Actuators:** LED street lamp driven by **PWM** through a MOSFET driver (brightness control); relay for an AC lamp.
- **Controller:** microcontroller with ADC + PWM, e.g. ATmega328P.

```
LOOP
   light = read LDR (ADC)
   IF light > DAY_LEVEL THEN           lamp OFF            (daytime)
   ELSE IF motion detected THEN        lamp 100 % for 60 s (someone is passing)
   ELSE                                lamp 30 %           (night, no traffic)
   use two thresholds (hysteresis) so the lamp does not flicker at dusk
END LOOP
```

Two improvements (evaluate = say the benefit and any cost):

1. **Motion-based dimming** (PIR + PWM): bright only when needed – saves about 40–60 % energy and extends LED life; small extra cost for the sensor.
2. **Solar panel + battery**: works during power cuts and needs no grid cable – very reliable; higher installation cost.

3. **Fault reporting** over GSM/LoRa: the lamp reports when it fails, so repairs are faster – improves reliability.

C. Smart home temperature monitoring (2026 Q1(c))

- **Inputs:** temperature sensor (DHT22 / LM35), humidity sensor, user set-point buttons or phone app.
- **Outputs:** LCD display, fan or air-conditioner through a relay, buzzer alarm, phone notification over Wi-Fi.
- **Processing unit:** ESP32 (Wi-Fi built in) or ATmega328P – reads the sensor, compares with the set point, controls the outputs.
- **Two constraints:** (1) **Real-time response** – the alarm must sound within seconds of overheating. (2) **Low power and low cost** – it runs all day every day, so it must use little electricity and be cheap enough for homes. (Others: accuracy ±0.5 °C, reliability.)

> **සිංහලෙන්:** Design ප්‍රශ්නයකට පිළිතුරු දෙද්දී හැමවෙලේම මේ headings 8 භාවිතා කරන්න: assumptions → sensors → controller → actuators → communication → power → logic (pseudocode) → challenges/improvements. හැම sensor එකකටම "ඇයි" කියන එකත් එක වාක්‍යයකින් ලියන්න (justify). Pseudocode එකේ IF/ELSE පැහැදිලිව තියෙන්න ඕනේ. "Evaluate" කියලා තිබ්බොත් වාසියයි, අවාසිය/වියදමයි දෙකම ලියන්න.

Past papers ▸ Answers: 2025 Q2(a)(b)(c) 25 marks · 2026 Q2(a)(b)(c) 25 marks · 2026 Q1(c) 11 marks · 2025 ICT Q2

---

12. Past paper map

Question	2025 Feb (and ICT 2025 – same paper)	2026 Mar
Q1	Definition + 3 characteristics (§1), 3 applications (§2), MCU vs MPU (§2)	Definition + characteristics (§1), functional vs non-functional (§3), temperature monitor (§11C)
Q2	Smart waste bin (§11A)	Smart street lighting (§11B)
Q3	Components (§4), sync vs async (§5), polling vs interrupts (§6)	AVR SUB + flags, DEC/BRNE loop (§9)
Q4	Functions in C, LED + button, ISR (§6, §7)	PORTB 0xAA, switch → LED (§7, §9)
Q5	Timers, 1-second delay, hard vs soft real-time (§8, §3)	Fetch–decode–execute, architecture data flow (§10)

Study notes written for your Semester 3.2 exam plan from the past papers in your folder. Check anything that looks different from what your lecturer taught.